

Model Checking

Software Quality Assurance

Dr. Abdul Rehman

Model Checking

- **Model checking** is a formal verification technique used in computer science and systems engineering to verify whether a system model meets a given specification.
- It is commonly applied to hardware and software systems to ensure correctness with respect to a certain set of properties, such as safety, liveness, and functional requirements.
- Model checking systematically explores the **states** of a system to verify that the system behaves as intended, making it a critical tool for identifying potential errors early in the development process.

Key components of Model Checking

- 1) **System Model**: This is a formal representation of the system under study. It is typically expressed as a state-transition system, describing states (possible configurations of the system) and transitions (changes between states due to actions or events).
- 2) **Specification**: The desired properties or behaviors of the system are specified in a formal language, often using temporal logics like Linear Temporal Logic (LTL) or Computation Tree Logic (CTL). Temporal logics allow properties to be expressed about sequences of states over time, such as "a request will eventually be granted" or "two processes will never enter a critical section at the same time."
- 3) **Model Checker**: A model checker is a tool or algorithm that automatically explores all possible states of the system model to verify whether the specified properties hold. If the model satisfies the specification, the model checker confirms correctness. If not, it provides a counterexample, which is a trace of states and transitions that demonstrate the property being violated.

How Model Checking Works

1. **State Exploration**: The model checker systematically examines all possible states and transitions in the system model to build a state space graph.
2. **Property Verification**: The model checker evaluates whether the specified properties hold true in all relevant states. Temporal logic is often used to express these properties, which can describe sequences of states or specific conditions over time.
3. **Counterexamples**: If a property is violated, the model checker generates a counterexample showing a sequence of states that lead to the violation. This trace helps developers understand why the property was not satisfied and where errors may exist.

Advantages of Model Checking

1. **Automated Verification**: Model checking is automated and systematic, eliminating the need for manual reasoning about all possible states.
2. **Early Bug Detection**: By identifying errors in models early, model checking can help avoid costly fixes later in the development process.
3. **Exhaustive Exploration**: Model checkers explore all potential states within the model's constraints, providing a high level of confidence in system correctness.

Limitations of Model Checking

1. **State Explosion Problem**: For complex systems with a large number of states, the state space can grow exponentially, making it challenging or infeasible for model checkers to explore all possibilities.
2. **Abstraction Required**: To manage complexity, system models may need to be simplified or abstracted, which can lead to oversights if important details are omitted.
3. **Specification Challenges**: Writing formal specifications using temporal logic can be difficult, and incorrect or incomplete specifications can lead to misleading verification results.

Applications of Model Checking (1/2)

1. **Hardware Verification**: Model checking is used extensively to verify circuit designs, such as microprocessors, for *correctness* and *performance* properties.
2. **Software Verification**: It helps verify critical software, particularly in safety-critical domains like *avionics*, *automotive systems*, and *medical devices*.
3. **Protocols Verification**: Model checking can verify network protocols and distributed systems to ensure properties such as *deadlock-freedom*, *mutual exclusion*, and *correct message delivery*.

Applications of Model Checking (2/2)

- In summary, model checking is a formal method that verifies whether a system model adheres to specified properties by exhaustively exploring all possible states and transitions.
- It is widely used in the design and verification of software, hardware, and complex systems to ensure reliability and correctness.

Model Checking (1/4)

The basic idea of what is known as *model checking* is to use algorithms, executed by computer tools, to verify the correctness of systems. The user inputs a description of a model of the system (the possible behavior) and a description of the requirements specification (the desirable behavior) and leaves the verification up to the machine. If an error is recognized the tool provides a counter-example showing under which circumstances the error can be generated. The counter-example consists of a scenario in which the model behaves in an undesired way.

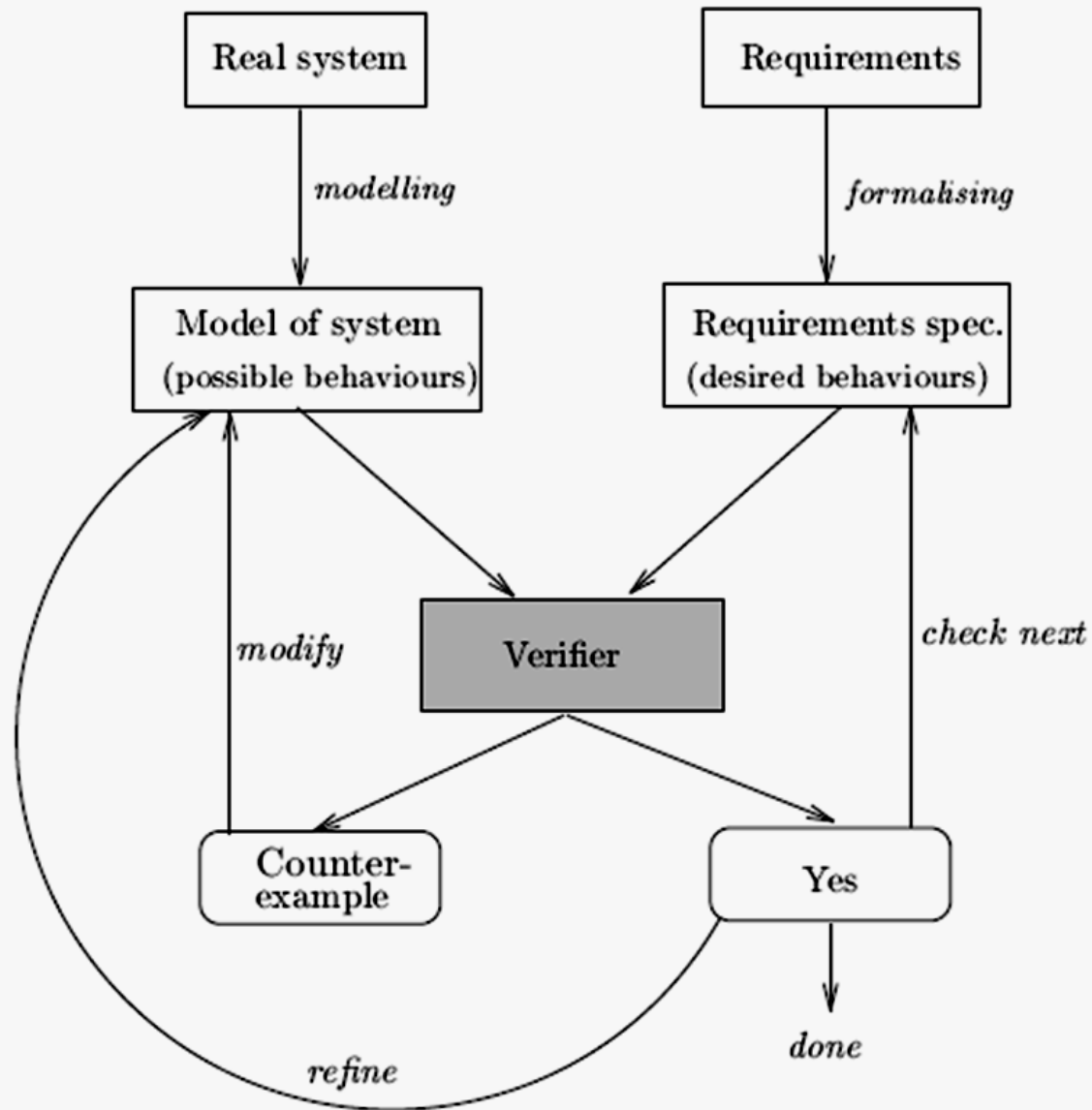


Figure : Verification methodology of model checking

Model Checking (2/4)

Thus the counter-example provides evidence that the model is faulty and needs to be revised. This allows the user to locate the error and to repair the model specification before continuing. If no errors are found, the user can refine its model description (e.g. by taking more design decisions into account, so that the model becomes more concrete/realistic) and can restart the verification process.

Model Checking (3/4)

The algorithms for model checking are typically based on an *exhaustive state space search* of the model of the system: for each state of the model it is checked whether it behaves “correctly”, that is, whether the state satisfies the desired property. In its most simple form, this technique is known as *reachability analysis*. E.g., in order to determine whether a system can reach a state in which no further progress is possible (a so-called deadlock), it suffices to determine all reachable states and to determine whether there exists a reachable state from which no further progress can be made. Reachability analysis is only applicable to proving freedom from deadlock and proving invariant properties, properties that hold during an entire computation.

Model Checking (4/4)

There has been quite some work done on automated reachability analysis for communication protocols in the late seventies and early eighties (West, 1982). Here, protocols were modeled as a set of finite-state automata that communicate via bounded buffered, asynchronous message passing. Starting from the initial system state which is expressed in terms of the states of the interacting automata and message buffers, all system states are determined that can be reached by exchanging messages. Protocols like X.21, X.25, and IBM SNA protocols have been analyzed automatically using this techniques. Model checking can in fact be considered as a successor of these early state-space exploration techniques for protocols. It allows a wider class of properties to be examined, and handles state spaces much more efficiently than these early techniques.

Model checking is an automated technique that, given a finite-state model of a system and a logical property, systematically checks whether this property holds for (a given initial state in) that model.

model checking is performed as an exhaustive state space search that is guaranteed to terminate since the model is finite.

*Model checking can provide a significant increase
in the level of confidence of a system.*

Methods of Model Checking

- There are basically two approaches in model checking that differ in the way the desired behavior i.e. the requirement specification is described
 - 1. Logic based or heterogeneous approach*
 - 2. Behavior based or homogeneous approach*

Logic based or heterogeneous approach

in this approach the desired system behavior is captured by stating a set of properties in some appropriate logic, usually some temporal or modal logic.

A system — usually modeled as a finite-state automaton, where states represent the values of variables and control locations, and transitions indicate how a system can change from one state to another — is considered to be correct with respect to these requirements if it satisfies these properties for a given set of initial states.

Behavior based or homogeneous approach (1/2)

in this approach both the desired and the possible behavior are given in the same notation (e.g. an automaton), and equivalence relations (or pre-orders) are used as a correctness criterion. The equivalence relations usually capture a notion like “behaves the same as”, whereas the pre-order relation represents a notion like “behaves at least as”. Since there are different perspectives and intuitions about what it means for two processes to “behave the same” (or “behave at least as”), various equivalence (and pre-order) notions have been defined.

Behavior based or homogeneous approach (2/2)

One of the most well-known notions of equivalence is bisimulation. In a nutshell, two automata are bisimilar if one automaton can simulate every step of the other automaton, and vice versa. A frequently encountered notion of pre-order is (language) inclusion. An automaton A is included in automaton B , if all words accepted by A are accepted by B . A system is considered to be correct if the desired and the possible behavior are equivalent (or ordered) with respect to the equivalence (or pre-order) attribute under investigation.

Although these two techniques are conceptually different, connections between the two approaches can be established in the following way.

the two approaches can be established in the following way. In particular a logic induces an equivalence relation on systems as follows: two systems are equivalent if (and only if) they satisfy the same formulas. Using this concept relationships between different logics and equivalence relations have been established. For instance, it is known that two models that are bisimilar satisfy the same formulas of the logic CTL, a logic that is commonly used for model checking purposes

The connection between the two different approaches is now clear: if two models possess the same properties (checked using the logic approach), then they are behaviorally equivalent (as could be checked in the behavioral approach).

The benefits of model checking (1/3)

- *General approach* with applications to hardware verification, software engineering, multi-agent systems, communication protocols, embedded systems, and so forth.
- Supports *partial* verification: a design can be verified against a partial specification, by considering only a subset of all requirements. This can result in improved efficiency, since one can restrict the validation to checking only the most relevant requirements while ignoring the checking of less important, though possibly computationally expensive requirements.

The benefits of model checking (2/3)

- Case studies have shown that the incorporation of model checking in the design process does not delay this process more than using simulation and testing. For several case studies the use of model checking has led to shorter development times. In addition, due to several advanced techniques, model checkers are able to deal with rather large state spaces (an example with 10^{130} states has been reported in the literature).
- Model checkers can potentially be *routinely used* by system designers with as much ease as they use, for instance, compilers; this is basically due to the fact that model checking does not require a high degree of interaction with the user.

The benefits of model checking (3/3)

- Rapidly increasing *interest of the industry*: jobs are offered where skills in applying model checkers are required; industry is building their own model checkers (e.g. Siemens and Lucent Technologies), have initiated their own research groups on model checking (e.g. Intel and Cadence) or is using existing tools (Fujitsu, Dutch Railways, to mention a few); finally, the first model checkers are becoming commercially available.
- *Sound and interesting mathematical foundations*: modeling, semantics, concurrency theory, logic and automata theory, data structures, graph algorithms, etc. constitute the basis of model checking.

The limitation of model checking (1/5)

The main limitations of model checking are:

- Appropriate mainly to *control-intensive applications* with communications between components. It is less suited to data-intensive applications, since the treatment of data usually introduces infinite state spaces.
- The applicability of model checking is subject to *decidability issues*: for particular cases — like most classes of infinite-state systems — model checking is not effectively computable. Formal verification, though, is in principle applicable to such systems.

The limitation of model checking (2/5)

- Using model checking a *model* of the system is verified, rather than the real system itself. The fact that a model possesses certain properties does not guarantee that the final realization possesses the same properties. (For that purpose complementary techniques such as systematic testing are needed.) In short,

Any validation using model checking is only as good as the model of the system.

The limitation of model checking (3/5)

- *Only stated requirements are checked:* there is no guarantee of the completeness of desired properties.
- Finding appropriate abstraction (such as the system model and appropriate properties in temporal logic) requires some expertise.
- As any tool, model checking software might be unreliable. Since (as we will see in the sequel of these notes) standard and well-known algorithms constitute the basis for model checking, the reliability does not in principle cause severe problems. (For some cases, more advanced parts of model checking software have been proven correct using theorem provers.)

The limitation of model checking (4/5)

- It is impossible (in general) to check generalizations with model checking (Apt and Kozen, 1986). If, for instance, a protocol is verified to be correct for 1, 2 and 3 processes using model checking, it cannot provide an answer for the result of n processes, for arbitrary n . (Only for particular cases this is feasible.) Model checking can, however, suggest theorems for arbitrary parameters that subsequently can be verified using formal verification.

The limitation of model checking (5/5)

Since in model checking it is quite common to model the possible behavior of the system as (finite-state) automata, model checking is inherently vulnerable to the rather practical problem that the number of states may exceed the amount of computer memory available. This is in particular a problem for parallel and distributed systems that have many possible system states — the size of the state space of such systems is in the worst case proportional to the product of the size of the state spaces of their individual components.

The problem that the number of states can become too large is known as the *state-space explosion problem*. Several effective methods have been developed to combat this problem;